

CP-SAT Production Optimizer

Approach, model formulation and comparison against the greedy scheduler

This document describes what the **optimizer/** package does, how the CP-SAT model is formulated, and how it differs from the greedy scheduler run by **local_main.py**. Every figure quoted is taken from the source on the **optimizer** branch or from measured output workbooks, not from estimates.

Scope

Area	Files
Model formulation	optimizer/model.py (1,114 lines)
Rolling-window driver + post-processing	optimizer/driver.py (993 lines)
Warm start from the greedy plan	optimizer/greedy_warmstart.py, optimizer/warmstart.py
Stage-1 carcass reschedule	optimizer/carcass_sched.py (441 lines)
Data loading (shared with greedy)	optimizer/data.py
Single config surface	optimizer/opt_config.py
Comparison baseline	b2c_pipeline.run_rolling_pipeline via local_main.py

Executive summary

- The optimizer solves the month as **3–4 rolling windows**. Each window commits 10 days and is solved over 13 days so the 3-day GT shelf seam is visible inside the window.
- It starts from the **greedy plan as a warm start**, so CP-SAT spends its time improving a known-good solution rather than hunting for a first feasible one.
- The objective is **maximise cured tyres**, with small penalties for curing changeovers and different-inch building changeovers.
- The single most important modelling decision is that curing is modelled as **press counts per SKU**, not per-press booleans. This collapses roughly 490,000 booleans to about 30,000 variables and is what makes a full month solvable.
- Several real plant costs are **deliberately not in the model** — same-size building changeovers, GT and carcass aging, and mould life. They are applied afterwards in a post-processing chain that only ever removes production.

Honest position on performance

On the same inputs, the greedy scheduler currently produces a **higher** plan than the CP-SAT optimizer: July 699,842 vs 653,844 cured, August 646,377 vs 615,993. The reason is not that CP-SAT is the wrong tool — it is that the model was thinned until it converged, and the post-processing that restores plant reality only subtracts. Section 9 sets out exactly where the gap comes from and what would close it.

1. Why CP-SAT and not LP, MILP or RL

The decisions the plant actually makes

Every scheduling decision on this line is a choice of *one* — one SKU on one machine, one mould pair on one press, one changeover taken or not taken. None of them is a “how much” question.

Approach	Why it does not fit	Evidence
LP	LP works in fractions. It would put 40% of machine 7001 on a 16" SKU and 60% on a 15" SKU. That plan cannot be run and it breaks the inch lock. A changeover becomes “0.3 of a changeover”, and the rule “run zero or at least 60 minutes” cannot be written at all.	The legacy LP in building.py has the machine assignment decided by a heuristic <i>before</i> the LP runs, and the minimum-campaign rule is a two-pass “add it, drop it if infeasible” hack.
MILP	MILP can represent the problem but cannot search it at this size. It works by solving the fractional version and branching, and the fractional version dodges every costly rule — so the bound sits far above any runnable plan and prunes almost nothing. On top of that, 170 curing presses are interchangeable, so branch-and-bound rediscovers the same plan with presses relabelled.	Building alone is ~350,000 yes/no decisions; per-press curing adds ~1.5 million more.
RL	The plan is released to the floor and must pass 19 hard feasibility rules. RL learns to avoid penalties; it cannot guarantee a press has 2 moulds or that Stage-2 has carcass. There is also no training data — the plant produces 12 plans a year, and each simulated month is the scheduler itself.	RL fits fast repeated dispatch under uncertainty, not one offline monthly plan.
GA / metaheuristics	Already tried and removed. No bound, no feasibility guarantee, slower convergence than a well-designed heuristic.	GeneticOptimiser was replaced by DemandHeuristicAssigner: “~100 lines vs ~600 lines of GA; deterministic, debuggable, demand-driven.”

What CP-SAT gives that the alternatives do not

- It decides *which, when and whether* directly, instead of inferring them from fractions.
- Hard plant rules enter as constraints, so any solution it returns is feasible in those rules by construction.
- It reports an optimality gap — greedy, GA and RL cannot tell you how far from the ceiling you are.
- It accepts a warm start, so the greedy plan you already trust becomes the floor.
- It supports a **deterministic** stop condition, which the cloud contract requires.

Caveat worth knowing

The current model uses **no CP-SAT logical constructs** — there is not a single `AddBoolOr`, `OnlyEnforceIf`, `AddCumulative`, `NewIntervalVar` or `AddNoOverlap` in `model.py`. All 61 constraints are plain linear with hand-written big-M. That means the model is essentially a MILP written inside CP-SAT, and is not yet using the reified and interval reasoning that is CP-SAT's main advantage over branch-and-bound.

2. How a run flows end to end

```
python -m optimizer.main
|
[1] opt_config.py          set month, horizon, demand file, all toggles
|
[2] data.load_model_inputs()  same ETL the greedy uses (DB + demand.xlsx)
|
[3] greedy_warmstart.extract_month_greedy()
    run the FULL greedy month once -> cached plan
|
[4] driver.run_rolling()    for each window:
    a. build the window warm-start hint, seam-aligned
    b. model.build_and_solve()  CP-SAT, 600 det-seconds
    c. commit the first 10 days, discard the 3-day tail
    d. carry GT / carcass / press counts / demand / CO budget forward
|
[5] driver._finalize_plan()  post-processing chain (Section 8)
|
[6] output_full.py          building + curing workbooks
```

Key design choice: the greedy is an input, not a rival

extract_month_greedy runs **b2c_pipeline.run_rolling_pipeline** for the whole month once, caches the result, and also writes it out immediately as **greedy_building_<date>.xlsx** / **greedy_curing_<date>.xlsx** so the greedy plan survives even if the optimizer run is long or interrupted. Every window is then warm-started from that plan.

Recorded reason for this design: a simple constructive warm start left each window under-solved and the month landed about 2% below the greedy engine. Warm-starting with the greedy engine's own plan reaches or beats it.

3. Decision variables

The relaxation that makes the month solvable

Curing is **not** modelled as “press P runs SKU S in shift T”. It is modelled as “**how many presses** run SKU S in shift T”. Because the 170 presses are largely interchangeable, per-press booleans would make the solver re-explore the same plan with presses relabelled. Counting eliminates that symmetry by construction.

Effect

About **490,000 booleans collapse to roughly 30,000 variables**. Which physical press ends up on which SKU is recovered afterwards as a bipartite matching in optimizer/writer.py, seeded with the previous window's setup so presses are not reshuffled at the seam.

Variable	Type	Indexed over	Meaning
$x[m,s,t]$	Bool	GT machine $\times$ SKU $\times$ shift	machine m is assigned to SKU s in shift t
$g[m,s,t]$	Int	same	GT units built, bounded by that machine's rate
xc / gc	Bool / Int	Stage-1 machine $\times$ SKU $\times$ shift	same pair for carcass
$n[s,t]$	Int	SKU $\times$ shift	presses assigned to SKU s
$co[s,t]$	Int	SKU $\times$ shift	presses newly changed over onto s
$prod[s,t]$	Int	SKU $\times$ shift	presses actually producing ($= n - co$)
built / cured / inv	Int	SKU $\times$ shift	GT reservoir flow
$carc_inv[s,t]$	Int	SKU $\times$ shift	carcass reservoir
$setup[m,i,t]$	Bool	flexible machine $\times$ inch $\times$ shift	which inch the machine is set up for
$dco[m,t]$	Int 0/1	flexible machine $\times$ shift	a different-inch changeover fires
$cmin[m,t]$	Int	flexible machine $\times$ shift	minutes lost to that changeover

What drives the variable count

- **Building pairs** $\times$ window shifts is the largest block. The historical inch lock is therefore the primary control on model size — it decides which (machine, SKU) pairs exist at all.
- **Curing SKUs** $\times$ shifts $\times$ 3 (n, co, prod) — note this scales with SKU count, not press count.
- A window is 13 days = **39 shifts**.

4. Objective function

The model **maximises**:

```
total_cured
- CO_PENALTY      * total_curing_COs
+ TERM_GT_W       * end_of_commit_GT
- inch_pen        (inch-relaxation COs, feature off by default)
- bld_co_pen      (different-inch building COs, per group)
- bld_rank_pen    (off-dominant inch preference, feature off)
- woff_pen        (aging write-offs, features off)
- smooth_pen      (JIT smoothing slack, features off)
```

Term	Weight	Notes
total_cured	1	Never scaled. Raising the scale to 10 was measured to lose about 11,000 cured units purely from worse convergence — large coefficients slow CP-SAT down.
CO_PENALTY	40 model default, 80 in opt_config	Cured-units charged per curing changeover. Keeps productive COs, cuts gratuitous churn.
TERM_GT_W	0 (off)	Weight on GT left at the commit boundary. This is the “value-to-go” term that stops rolling windows front-loading. It is currently disabled.
bld_co_pen	per group, see below	Different-inch building changeovers only. Same-inch changeovers carry no penalty and no capacity charge in the model.

Per-group different-inch changeover penalty

Group	Penalty (model)	Penalty (opt_config)	Monthly budget per machine	Rationale
Stage-2	0	0	uncapped	GT bottleneck — inch freedom is worth more than the CO cost
VMI	4	12	4	light — the flexible group
BJ	20	12	2	heavy — stays on its dominant inch for high utilisation
Unistage narrow	20	20	2	heavy
Stage-1	20	20	2	heavy

The penalty shapes the choice; the **monthly budget is the hard control**. The driver carries each machine's remaining allowance across windows, so a machine cannot spend its whole month's inch changes in window 1.

Reading the objective correctly

The objective is expressed in **cured units**. A curing changeover priced at 80 means the solver will take it only if it expects to gain more than 80 cured tyres. This is why the coefficients must stay small — they are competing directly with the quantity being maximised, and inflating them degrades search rather than expressing a stronger preference.

5. Constraints

Building

Rule	Form
A machine runs at most one SKU per shift	$\sum_s x[m,s,t] \leq 1$
Production requires assignment	$g \leq \text{rate} \times x$
Different-inch CO costs capacity	$480 \times g \leq \text{rate} \times (480 - \text{cmin})$ when assigned
Inch setup persists through idle shifts	setup state carries when the machine is idle, so a machine cannot dodge the CO cost by idling one shift
Hard monthly diff-CO budget	$\sum_t \text{dco}[m,t] \leq \text{remaining allowance}$

Curing

Rule	Form
Changeover detection	$\text{co}[s,t] \geq n[s,t] - n[s,t-1]$
A changing press produces nothing	$\text{prod} = n - \text{co}$
Press roster	$\sum_s n[s,t] \leq 170$
Daily changeover cap	$\sum \text{co}$ over the 3 shifts of a day ≤ 12
Mould pair availability	$n[s,t] \leq \min(\text{mould pairs for } s, \text{eligible presses for } s)$
Shared-mould contention	SKUs that share moulds are grouped by union-find; each group's total presses $\leq \lceil \text{shared moulds} / 2 \rceil$

Inventory, shelf life and storage

Rule	Form
GT flow	$\text{inv}[s,t] = \text{inv}[s,t-1] + \text{built} - \text{cured}$
GT shelf life 3 days	$\text{inv}[s,t] \leq \text{sum of GT built in the last 9 shifts}$
GT storage cap	$\sum_s \text{inv}[s,t] \leq 8,000$
Carcass flow	$\text{carc_inv} = \text{previous} + \text{built} - \text{consumed by Stage-2}$
Stage-2 waits for carcass	$\text{carc_inv} \geq 0$ is what forces this — Stage-2 can never build GT the carcass cannot back
Carcass shelf life 1 day	$\text{carc_inv}[s,t] \leq \text{carcass built in the last 3 shifts}$
Carcass storage cap	$\sum_s \text{carc_inv}[s,t] \leq 1,200$

Demand

Rule	Form
Sacred demand cap	$\sum_t \text{cured}[s,t] \leq \text{demand}[s]$
No overbuild	$\sum_t \text{built}[s,t] \leq \text{demand}[s] - \text{opening GT}[s]$

Not modelled as constraints

Mould life and mould cleaning do not exist in the model at all. The variables are loaded but never referenced. The model's throughput is therefore optimistic by roughly the 50–65 cleaning shifts per month the plant actually pays, and that is deducted afterwards.

6. Rolling horizon window

```
Window 1: solve days 1-13 -> COMMIT days 1-10 (days 11-13 discarded)
Window 2: solve days 11-23 -> COMMIT days 11-20
Window 3: solve days 21-33 -> COMMIT days 21-30
Window 4: remainder
```

```
COMMIT_DAYS      = 10      LOOKAHEAD_TAIL = 3      MAX_WINDOW_DAYS = 13
```

Why a 3-day lookahead tail

GT has a 3-day shelf life. Without a tail, the model would treat the last committed shift as the end of the world and would happily build GT it can never cure. The tail makes the shelf seam visible inside the window. It is solved and then thrown away.

What carries across the seam

Carried state	Purpose
GT inventory per SKU	opening stock for the next window
Carcass inventory per SKU	opening stock for the next window
Running press count per SKU	so the next window does not re-pay changeovers already made
Physical press → SKU setup	so the bipartite recovery does not reshuffle presses at the seam
Press → mould assignment	mould continuity
Building machine → last SKU	so a seam SKU change is charged a building changeover
Remaining diff-CO budget per machine	monthly budget cannot be spent twice
Remaining demand	shrunk by realised cured, not model cured — so the demand cap holds across the whole month

Solver settings

Setting	Value	Note
Time per window	600 deterministic seconds	~30–40 minutes for a full month
Workers	8	
Stop condition	deterministic only when workers ≤ 1	At the shipped default of 8 workers the run is wall-clock bounded and therefore not reproducible . Set workers = 1 for any run that must be repeatable.
Warm start	repair_hint = True, fix_to_hint = False	The hint is a floor, not a cage

Two known configuration defects

(1) opt_config exposes OPT_COMMIT_DAYS and OPT_LOOKAHEAD, but driver.py hard-codes 10 and 3 — changing them in opt_config does nothing. (2) driver defaults OPT_SAME_CO_TRIM, OPT_CARC_RESCHED and OPT_AGE_TRIM to 0, while opt_config sets them to 1. Any entry point that does not import opt_config — run_month_full.py, for example — produces a materially different plan.

7. Warm start

What is hinted

Only two variable families are hinted: **n[s,t]** (press counts) and **g[m,s,t]** (GT built). Everything else is **derived** inside the model so CP-SAT receives a complete, self-consistent incumbent.

```
co      = max(0, n - previous_n)
prod    = n - co
cured   = cure_rate * prod
built   = sum of g over machines
inv     = running reservoir, clamped to [0, 8000]
gc/xc   = a same-shift carcass pack covering the hinted Stage-2 GT
```

Recorded reason: a **partial** hint left the solver returning UNKNOWN or zero at single-worker — measurably worse than a cold start. Completing the hint fixed it.

GT-cap trim on the hint

The greedy banks more GT than the model's 8,000 cap allows, so the raw greedy plan is infeasible as a hint. The code simulates the reservoir shift by shift and, wherever the running total exceeds the cap, reduces the hinted build on the **highest-inventory SKUs first**. This is stricter than the model's own per-shift cap.

Warm-start modes

Mode	Behaviour	Status
greedy	Seam-aligned replay of the real greedy plan through this window's actual carry-over state	Shipped (opt_config default)
simple	Constructive per-window greedy seeded from month-opening state	Seam-blind for later windows
greedy_slice	Raw index slice of the greedy month plan	Measured infeasible at the seam. Marked “ do not ship ” in the source.
none	All-zero seed, pure CP-SAT	Baseline for A/B

Why the seam-aligned replay is necessary

Window 2 starts from CP-SAT's committed state, which cures less than greedy did. A raw slice of the greedy plan is therefore not jointly feasible against that state — the changeover count can breach the daily cap, and cured can exceed opening GT plus built, driving inventory negative. The replay re-runs the greedy logic forward through the *actual* carry-over instead of copying its numbers.

8. Post-processing — where the model is brought back to reality

Several real costs are outside the model. They are applied after the solve, in this order. Every step in this chain **only removes production**.

#	Step	What it does	Effect on cured
1	Same-size building CO trim	Walk each machine's producing shifts. Where the SKU changed but the inch did not, charge the same-size CO minutes and cut the shift's quantity to fit 480 minutes. Excess is redistributed to continuation shifts with spare capacity, clamped by each SKU's remaining demand. Only the unplaceable remainder is lost.	Reduces
2	Mould clean	Replay mould life per press from the DB opening value, 3,000 cycles. At zero, idle that shift for a 480-minute clean and reset. A curing changeover also resets life, because the CO includes a clean.	Reduces
3	Aging write-off	Per-SKU FIFO over calendar days. GT older than 3 days and carcass older than 1 day are removed from the building rows. Iterated to a fixed point, because removing stock reshuffles the FIFO.	Neutral — only never-consumed stock is removed
4	Stage-1 carcass reschedule	See Section 9. Discards the model's carcass timing entirely and rebuilds it just-in-time.	Reduces (cuts Stage-2 GT that carcass cannot back)
5	Cured cascade	Any SKU where cured now exceeds opening GT plus built has its curing rows cut, latest shift first.	Reduces

Why same-size changeovers are outside the model

Measured, not assumed

Charging same-size changeovers inside the model requires per-SKU continuation binaries. That was built and measured: it **wrecked the CP-SAT gap and cost around 90,000–96,000 cured units**. So the cost was moved to post-processing. The trade is explicit in the source: the model plans as if same-size changeovers are free, and roughly **14,000 units** are then cut back out. This single item is the largest identified loss in the optimizer path.

The same pattern applies to aging

In-model aging constraints were also built and measured, and cost roughly **80,000 units** through worse convergence. They too were moved to post-processing.

9. Stage-1 carcass — decoupled just-in-time reschedule

Why the model's carcass schedule is thrown away

The CP-SAT model schedules carcass **front-loaded**: it banks carcass early, but the Stage-2 GT that consumes it runs more than a calendar day later. That violates the plant's hard 1-day carcass shelf. So **carcass_sched.reschedule_carcass** keeps every GT decision the model made, deletes every carcass row, and rebuilds Stage-1 from scratch.

How the rebuild works

Step	Logic
Demand	Carcass demand = Stage-2 GT, one for one, keyed by shift
SKU order	Most-constrained first — SKUs with the fewest eligible Stage-1 machines claim their scarce machines before more flexible SKUs can take them
Timing	Same shift first, then spill back at most one calendar day . Never earlier.
Opening carcass	Covers the earliest consumption first, mirroring the validator's FIFO
Machine order	Cheapest changeover first — continuation 0, same-inch 60, different-inch 180 — so carcass campaigns stick to one machine
Capacity check	Enforces production + CO ≤ 480 and one SKU per machine per shift, and re-evaluates the successor slot's changeover before inserting. Rejects the insert if the successor would overflow.
FIFO reconcile	Replays the validator's exact per-SKU FIFO. Where it cannot serve a Stage-2 GT unit from carcass aged ≤ 1 day it reduces the Stage-2 GT row in place and drops the stranded carcass.

This is the strongest single idea in the optimizer package

The FIFO reconcile makes the carcass rules hold **by construction** at every shift, rather than being checked afterwards and hoped for. It achieves this by cutting Stage-2 GT — so it lowers the KPI and makes it honest at the same time. The greedy pipeline does not have this, nor the successor-aware capacity check, nor most-constrained-first SKU ordering. All three are worth porting.

What the greedy has that this does not

- **A month-level inch lock for Stage-1.** The greedy pins each Stage-1 machine to one inch for the whole month, solved upfront from monthly carcass demand capped by curing throughput. The optimizer decides shift by shift, so a machine can bounce between inches at 180 minutes each.
- **Deeper pre-build.** The greedy banks up to 2 shifts ahead against a known Stage-2 rate; the reschedule only spills back 1 day from actual consumption.
- **Changeovers charged during the decision.** In the greedy, Stage-1 CO time reduces capacity inside the gate, so Stage-2 is clamped in the same shift. Here the cost lands after Stage-2 is already fixed, so the only remedy is cutting Stage-2.

10. Comparison with the greedy scheduler

Measured results, same inputs

Month	Demand	Greedy cured	CP-SAT cured	Difference
July 2026	727,779	699,842 (96.16%)	653,844 (89.84%)	greedy +45,998
August 2026	689,563	646,377 (93.74%)	615,993 (89.33%)	greedy +30,384

July figures are from the output workbooks. The August CP-SAT figure is the value recorded in `opt_config.py`; the August optimizer workbook on disk is a partial run.

Structural differences

Dimension	Greedy (<code>local_main.py</code>)	CP-SAT optimizer
Decision scope	One shift at a time, no lookahead beyond the buffer horizon	10 committed days at a time, with a 3-day tail
Optimality	None — first good choice wins	Reports a gap against a bound
Curing changeovers	Planned upfront for the month, plus reactive changes	Decided inside the optimisation, priced against cured units
Press assignment	Explicit per press throughout	Relaxed to counts, recovered afterwards by bipartite matching
Same-size building CO	Charged inside the decision — reduces capacity in the same shift	Not modelled; trimmed afterwards
Aging and mould life	Modelled inside the simulation	Not modelled; deducted afterwards
Stage-1 inch	Locked for the month , solved upfront	Decided per shift
Carcass feasibility	Gate clamps Stage-2 in the same shift	FIFO reconcile guarantees it by construction
Runtime	A few minutes	30–40 minutes
Reproducibility	Deterministic	Deterministic only at workers = 1

Where each one is genuinely stronger

Greedy is better at	CP-SAT is better at
Paying every real cost as the decision is made, so its number is already honest	Seeing 10 days at once instead of one shift
Month-level Stage-1 inch planning → long campaigns, few changeovers	Pricing a changeover against the cured units it buys, rather than a fixed rule
Speed and reproducibility	Guaranteeing carcass feasibility by construction
No repair pass — what it plans is what it reports	Reporting how far the plan is from the ceiling

11. Why CP-SAT currently loses, and what would close the gap

Root cause in one sentence

The model optimises an easier problem than the plant has, and the repair chain that restores reality only subtracts.

Same-size changeovers, GT and carcass aging, and mould life are all outside the model. The solver produces a plan that ignores them, and post-processing then removes production to put them back. The greedy pays those costs inside its own simulation, so its number is already honest. The two are not being compared on equal terms — and the repair direction is always downward.

Everything that was tried in-model and measured worse

Attempt	Measured	Outcome
Same-inch CO as continuation binaries	–96,000 / –90,000	Moved to post-processing
GT and carcass aging as constraints	about –80,000	Moved to post-processing
Full DB-allowable inch flexibility	August 542k vs 616k	Forced to the narrow historical lock
Targeted scarce-inch flexibility (13"/15")	601,889 vs 615,993 (–14k)	Left off
Blanket inch-lock relaxation	about –11%	Left off
Priority scale raised to 10	about –11,000	Kept at 1
Minimum campaign floor in-model	breaks the warm start	Left off

This record matters: the thin model was not laziness. Every richer formulation was built, measured and rejected because CP-SAT stopped converging. The source records “the 89.33% is a hard attractor; inch allocation is not the lever — the ~14k same-CO post-process cut is the real loss.”

The four highest-value moves, in order

#	Move	Why
1	Fix the toggle-default divergence	driver defaults SAME_CO_TRIM / CARC_RESCHED / AGE_TRIM to 0 while opt_config sets them to 1. Two entry points currently produce different plans. Until this is fixed, comparisons are unreliable.
2	Set workers = 1 for any measured run	At 8 workers the stop is wall-clock, so results are not reproducible and A/B tests are noise.
3	Enable TERM_GT_W	It is 0 today, so the model has no reason to leave GT for the next window. This is the textbook receding-horizon end-effect and it is a free experiment.
4	Approximate same-size CO cost inside the model	Not with continuation binaries — that is proven to cost 90k+. Instead charge an estimated same-CO minute cost on any shift where a machine is assigned a SKU different from its window-entry SKU. No new binaries, and it removes most of the ~14k repair.

A fifth, longer-term move

The model uses no CP-SAT logical constructs at all. Expressing the setup and changeover logic with reified constraints, and machine occupancy with interval and no-overlap constraints, is exactly where CP-SAT beats a MILP formulation. The current model is not yet using the engine it runs on.

12. Configuration reference

Everything is set in **optimizer/opt_config.py**, which pushes values into the environment variables the other modules read. A real shell environment variable still overrides it, for A/B tests.

Group	Setting	Value	Meaning
Plan	MONTH_YEAR / MONTH_NUM / MONTH_DAYS	2026 / 8 / 32	Drives plan_month for opening GT, carcass and the running-moulds snapshot
Plan	MONTH_DEMAND	correct_july.xlsx	Demand workbook under data/input/
Solve	OPT_DET_TIME	600	Seconds per window
Solve	OPT_WORKERS	8	Set to 1 for reproducible runs
Solve	WARMSTART	greedy	Seam-aligned replay of the greedy plan
Solve	OPT_CO_PEN	80	Cured units charged per curing changeover
Curing	OPT_PRESS_EFF	0.94	Cure rate = $\text{int}(\text{int}(480/\text{CT}) \times 2 \times \text{eff})$
Curing	OPT_MAX_CO_PER_DAY	12	Curing changeovers per calendar day
Caps	GT_CAP_MAX	8,000	Total GT held per shift
Caps	CARC_CAP_MAX	1,200	Total carcass held per shift
Inch	OPT_INCH_FLEX	0	Narrow historical inch sets. 1 makes the solver drown — keep 0.
Inch	OPT_TARGET_FLEX	empty	Targeted scarce-inch flex measured a regression
Post	OPT_SAME_CO_TRIM	1	Charge same-size building CO after the solve
Post	OPT_AGE_TRIM	1	GT 3-day and carcass 1-day write-off
Post	OPT_CARC_RESCHED	1	Just-in-time Stage-1 reschedule
Post	OPT_MOULD_CLEAN	1	8-hour clean at 3,000 cycles
Post	OPT_BUILT_CAP	1	Built $\leq$ demand per SKU (in-model constraint)

Physical constants

Constant	Value
Shift length / shifts per day	480 minutes / 3 (A 07:00, B 15:00, C 23:00)
GT shelf life	3 days
Carcass shelf life	1 day
Curing changeover	480 minutes — one full shift
Mould clean	3,000 cycles, then 480 minutes
Cavities per press	2 tyres per cycle

How to run it

```
cd /Users/anmolsaini/Documents/cbc
myenv/bin/python -m optimizer.main

# for a reproducible measured run:
OPT_WORKERS=1 myenv/bin/python -m optimizer.main
```

Prepared from source on the **optimizer** branch and from measured output workbooks. Where the source records a measurement, that figure is quoted rather than estimated.